

ReChisel: Effective Automatic Chisel Code Generation by LLM with Reflection

Juxin Niu¹, Xiangfeng Liu^{1,2}, Dan Niu³, Xi Wang^{4,5*}, Zhe Jiang^{4,5}, Nan Guan^{1*}

¹ Department of Computer Science, City University of Hong Kong, Hong Kong SAR

² School of Computer Science and Engineering, Northeastern University, China

³ School of Automation, Southeast University, China

⁴ National Center of Technology Innovation for EDA, China ⁵ School of Integrated Circuits, Southeast University, China

Email: juxin.niu@my.cityu.edu.hk, xiangfengliu.cn@gmail.com, 101011786@seu.edu.cn, xi.wang@seu.edu.cn,

zhejiang.arch@gmail.com, nanguan@cityu.edu.hk

*Corresponding authors.

Abstract—Coding with hardware description languages (HDLs) such as Verilog is a time-intensive and laborious task. With the rapid advancement of large language models (LLMs), there is increasing interest in applying LLMs to assist with HDL coding. Recent efforts have demonstrated the potential of LLMs in translating natural language to traditional HDL Verilog. Chisel, a next-generation HDL based on Scala, introduces higher-level abstractions, facilitating more concise, maintainable, and scalable hardware designs. However, the potential of using LLMs for Chisel code generation remains largely unexplored. This work proposes ReChisel, an LLM-based agentic system designed to enhance the effectiveness of Chisel code generation. ReChisel incorporates a reflection mechanism to iteratively refine the quality of generated code using feedback from compilation and simulation processes, and introduces an escape mechanism to break free from non-progress loops. Experiments demonstrate that ReChisel significantly improves the success rate of Chisel code generation, achieving performance comparable to state-of-the-art LLM-based agentic systems for Verilog code generation.

I. INTRODUCTION

In hardware design, coding with hardware description languages (HDLs) like Verilog is often a time-consuming and labor-intensive task. Large language models (LLMs) have demonstrated promising capabilities in assisting the coding processing in not only software but also hardware design. Recently, LLMs have been employed for translating natural language to HDL code [11], [25], assisting with debugging [27], [33], and optimizing code implementation to improve key chip metrics such as performance, power, and area (PPA) [34]. These efforts primarily focus on traditional HDLs like Verilog.

Chisel [2] is a next-generation HDL embedded in Scala that facilitates advanced circuit design. Unlike Verilog, which primarily targets the Register Transfer Level (RTL) and often necessitates detailed low-level specifications, Chisel offers higher-level abstractions through object-oriented and functional programming features. This approach results in more concise, maintainable, and scalable designs [10]. Due to its compatibility with agile design methodologies, Chisel is gaining popularity for complex hardware design [13], including processors (e.g., XiangShan [31], BOOM [36], and RocketChip [1]), accelerators (e.g., MAGMA-Si [6] and Gemmini [8]), among many others.

However, the potential of using LLMs for Chisel code generation remains largely unexplored. As a relatively new language, Chisel has significantly less publicly available code online—for instance, the amount of Chisel code on GitHub is only about 1/20th that of Verilog—which limits the training data available for LLMs to generate Chisel code effectively. To better understand these limitations, we conducted experiments to evaluate the baseline performance of mainstream LLMs in automatically generating Chisel code in a zero-shot setting. The experiment results show that the baseline capabilities of these LLMs are indeed limited.

In this paper, we propose ReChisel, an LLM-based agentic system designed to improve the effectiveness of using LLM for Chisel code generation. ReChisel incorporates a reflection mechanism, utilizing an iterative process to refine the quality of generated code. The key insight is leveraging feedback from compilation and simulation processes to guide the LLM in correcting errors. Additionally, we observed that during the reflection process, the LLM may enter a non-progress loop. To overcome this, we developed an escape mechanism that enables the LLM to break free from these loops and resume making useful attempts. We evaluate the performance of ReChisel on three benchmarks, VerilogEval's Spec-to-RTL [21], AutoChip's HDLBits [9] and RTLLM [17], with five mainstream LLMs (various versions of GPT-4 and Claude 3.5). The experimental results show that our method significantly improves the success rate of generating Chisel code. Additionally, we compare the performance of LLMs in generating Chisel versus Verilog code. Although the baseline capabilities of LLMs are significantly weaker for Chisel code generation than for Verilog, the performance of ReChisel reaches a level comparable to the state-of-the-art LLM-based agentic systems for Verilog code generation. This not only validates the effectiveness of our proposed method but also highlights the potential of Chisel in LLM-assisted hardware design. ReChisel is open-sourced at <https://github.com/niu-juxin/rechisel>.

II. BACKGROUND & RELATED WORK

A. LLM-assisted Hardware Design

In hardware design, coding with hardware description languages (HDLs) like Verilog is a time-consuming and labor-intensive task. Recent advancements in LLMs have demonstrated transformative potential in diverse domains, sparking increasing interest in leveraging LLMs to streamline and improve the hardware design process.

Some research [4] aims at automating the end-to-end hardware design process using LLMs. ChatCPU [28] introduces the first agile hardware design and verification platform, and develops a 6-stage pipelined RISC-V CPU prototype with the platform. Other studies [3], [20], [25], [30] focus on enhancing the effectiveness of HDL code generation through LLMs. RTLcoder [16] presents an agent-based framework for automatically generating labeled datasets, which are then used to fine-tune a lightweight model that achieves performance comparable to GPT models. RTLrewriter [34] leverages LLMs for code optimization to enhance key chip metrics such as power, performance, and area (PPA). Some work [24] emphasizes assessing the performance of LLMs in hardware design tasks. RTLLM [17], [19] provides an open-source RTL generation benchmark consisting of 50 design problems that span a diverse and broad

TABLE I: Comparison of LLM baseline capabilities in generating Chisel and Verilog code.

Model	Pass@1 (%)		Pass@5 (%)		Pass@10 (%)	
	CHS	VRL	CHS	VRL	CHS	VRL
GPT-4 Turbo	45.54	67.61	61.97	77.46	66.20	81.22
GPT-4o	45.07	69.48	65.26	75.59	70.89	77.46
GPT-4o mini	11.27	59.15	28.64	69.48	36.62	72.30
Claude 3.5 Sonnet	33.33	77.93	52.58	82.16	59.62	84.04
Claude 3.5 Haiku	26.29	75.59	54.46	83.57	58.69	84.04

* CHS and VRL represent Chisel and Verilog, respectively.

spectrum of complexities. VerilogEval [15], [21] provides benchmarks for two tasks: code completion and translating specifications to RTL. MG-Verilog dataset [35] divides module implementations into multiple abstraction levels, enabling the evaluation of LLM capabilities across varying degrees of abstraction. These works focus on Verilog, leaving the effectiveness of using LLMs for Chisel code generation an open question yet to be explored.

B. Chisel: Next-Generation HDL

Chisel [2] is a next-generation HDL embedded in Scala, leveraging Scala’s advanced syntax and features to enable high-level circuit design. Unlike traditional HDLs like Verilog, which focus on register-transfer level (RTL) and require detailed low-level specifics, Chisel introduces modern programming paradigms such as object-oriented design (OOD) and functional programming. This allows for modular, reusable, and parameterized hardware design [13]. Chisel is gaining popularity for complex hardware design [13], including processors (e.g., XiangShan [31], BOOM [36], and RocketChip [1]), accelerators (e.g., MAGMA-Si [6] and Gemmini [8]), among many others. ChatChisel [18] leverages LLMs to generate Chisel code, designing and implementing a RISC-V CPU with a five-stage pipeline and branch prediction. This work shows the advantages of using LLMs with Chisel in complex hardware design scenarios, highlighting Chisel’s potential of using LLM to assist in hardware generation.

Chisel is not directly used for hardware implementation; instead, it serves as a high-level language that must first be *compiled* into Verilog [22]. The compilation process begins with the execution of Chisel code, using the Chisel library to construct a hardware design graph. This design is then serialized into a platform-independent intermediate representation known as FIRRTL [12], [14]. The FIRRTL representation is optimized and ultimately transformed into Verilog code through the FIRRTL compiler.

C. Reflection

Reflection [23], [29], [32] enables LLMs to self-analyze their reasoning processes and actions, dynamically adapting to enhance reliability, adaptability, and effectiveness. In the context of code generation, reflection allows LLMs to automatically identify syntax errors or logical flaws in generated code. AutoChip [26] leverages reflection to improve the performance of Verilog code generation. VerilogCoder [11] introduced an autonomous agent capable of performing graph planning, and integrates an AST-based waveform tracing tool to provide feedback for Verilog code debugging.

III. MOTIVATION

Chisel’s high-level abstractions and modern programming principles make it well-suited for agile hardware design and increasingly popular for RTL design. However, it remains unclear whether Chisel is also a viable choice for automatic RTL code generation using LLMs. Should we use LLMs to directly generate Verilog code from a natural language specification, or should we generate Chisel code

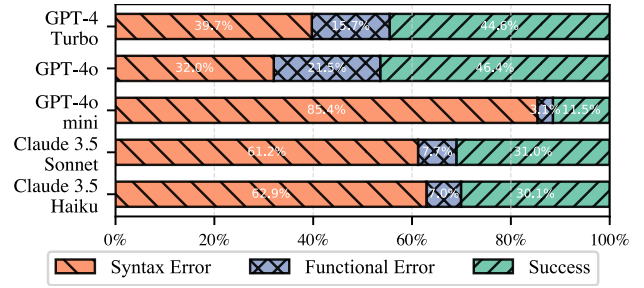


Fig. 1: Proportion of different error types in generated Chisel code.

and then compile it into Verilog? As a first step towards answering this question, this paper explores the capability of LLMs to generate *correct* Chisel code.

Compared to Verilog, Chisel presents both potential advantages and disadvantages for LLM-based automatic code generation. On the downside, Chisel is relatively new and has significantly less publicly available code for model training—approximately only 1/20th the volume of Verilog code on GitHub¹. On the upside, Chisel’s similarity to software languages and its incorporation of modern programming concepts enables LLMs to focus on higher-level logic in complex scenarios rather than low-level specifications. Additionally, the strong capabilities of LLMs in handling software code like Java could potentially transfer to Chisel code generation. Given these factors, it remains uncertain whether Chisel can be a competitive choice for automatic RTL code generation using LLMs.

As a preliminary step toward understanding LLMs’ performance in Chisel code generation, we assessed mainstream LLMs’ *baseline* capabilities in generating Chisel code, and compared it with the more widely studied Verilog to identify potential performance gaps. The evaluation was performed under *zero-shot* conditions, providing the LLMs with only the specification without additional context. Each model was tasked with producing *complete* module implementations in either Verilog or Chisel in a *single* pass. We evaluate using the widely adopted Pass@*k* [5] metric, which measures the probability of achieving at least one correct result among the top *k* generated attempts. Results in Table I show that the ability of LLMs in HDL code generation to use Chisel is significantly less effective than Verilog. We further analyze the proportions of different types of errors that occur when LLMs generate Chisel code. As illustrated in Fig. 1, a significant proportion of errors arise during the compilation phase, which indicates that these LLMs still struggle with ensuring the syntactic correctness of Chisel code. This huge performance gap highlights the necessity of exploring new methods to enhance LLMs’ ability to generate Chisel code.

IV. RECHISEL DESIGN

A. Overview

We introduce ReChisel, an LLM-based agentic system designed to enhance the effectiveness of Chisel code generation. Fig. 2 illustrates the workflow of ReChisel. Given a specification and a testbench, the goal is to generate correct Chisel code that passes all tests.

The workflow begins with the *Generator* creating Chisel code based on the specification (**Step 1**). The *Compiler* then translates this code into Verilog (**Step 2**), followed by functional testing of the Verilog code using the *Simulator* (**Step 3**). If both compilation and

¹As of March 2025, our GitHub advanced search revealed 1.60 million lines of publicly available Verilog/SystemVerilog code and 80.1 thousand lines of Scala code importing Chisel extensions.

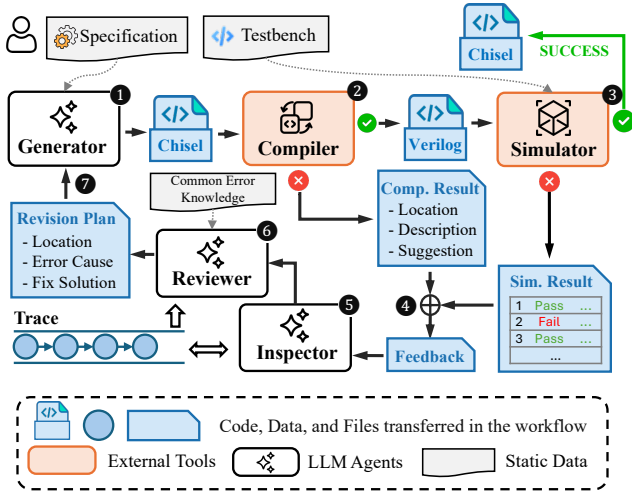


Fig. 2: ReChisel workflow: ReChisel is an LLM-based agentic system designed to enhance the effectiveness of Chisel code generation.

simulation succeed, the Chisel code is considered correct, concluding the workflow, and the user receives the validated code. However, if errors occur, ReChisel employs a reflection mechanism to iteratively generate code until it succeeds. In each iteration, results are collected and organized to provide feedback to the *Inspector* (Step 4). The *Inspector* maintains a trace that includes all previous iterations. Upon receiving the feedback, the *Inspector* updates the trace (Step 5). The *Reviewer* then analyzes the trace to formulate a revision plan (Step 6), which is passed back to the *Generator* as guidance for error correction. The *Generator* applies the necessary code adjustments according to the plan to produce a new version of the Chisel code (Step 7). The updated code re-enters the workflow, initiating the next iteration. The *Generator*, *Reviewer*, and *Inspector* are LLM agents whose roles in the workflow are defined by the provided system prompts, while the *Compiler* and the *Simulator* are external tools.

A *maximum number of iterations* is set to limit the number of attempts the LLM can make. If the number of iterations exceeds this limit, the workflow terminates, and the case is considered a failure. The number of iterations is a key factor affecting the effectiveness of ReChisel. In general, a higher number of iterations typically leads to better performance. However, the performance gains from increasing the number of iterations is limited by the inherent capabilities of the LLM. In Section V, we provide a detailed analysis of how the number of iterations impacts the effectiveness of ReChisel.

To enhance the reflection process, we propose different feedback strategies for different types of results. We also adopt in-context learning methods for more effective reviews. These approaches are detailed in Section IV-B. Additionally, we observed that the LLMs may fall into a *non-progress loop* during reflection. To address this issue, we propose an *escape mechanism* implemented within the *Inspector*. The detailed discussion is given in Section IV-C.

B. Error Feedback & Reflection Mechanism

Errors in testing LLM-generated Chisel code fall into two types: *syntax errors* reported in compilation and *functional errors* discovered in simulation. We propose different feedback strategies for each type.

Syntax Error. During the compilation process, the compiler conducts a series of checks to ensure that the code is syntactically correct and can be successfully translated. When errors are detected, the compiler reports the specific location of each issue, provides a detailed explanation of the error and, in some cases, suggests possible

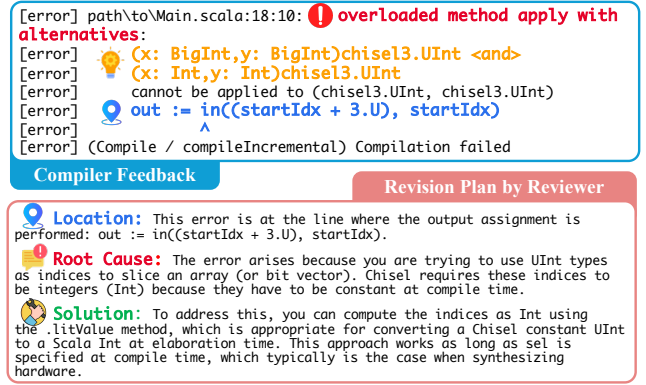


Fig. 3: Example of compiler feedback and revision plan.

fixes. This information is organized into an error list and serves as feedback. Fig. 3 illustrates an example of compiler feedback.

We found that the types of syntax errors tend to be densely distributed. Specifically, the most common errors generated by the LLM include mixing Scala and Chisel syntax, handling signal types, and managing clock domains. In Table II, we show several examples of common errors made by the LLM along with the corresponding feedback provided by the compiler. Beyond syntax errors, we found that the compiler performs static code analysis during compilation and reports logical errors that can cause functional errors in simulations. For instance, the compiler can detect uninitialized or partially initialized signals (B3), preventing unintended latch generation in hardware. Besides, it can identify combinational loops (C2) to avoid unpredictable signal oscillations. For simplicity, we also refer to these logical errors detected at compile time as syntax errors in the future discussion. For these common errors, we employ in-context learning [7] to further enhance the effectiveness of reviews. We pre-organized the causes and corresponding fix guidance for each error and included this information within the prompts provided to the LLM. This approach enables the LLM to deliver feedback more quickly and accurately when encountering such errors.

Functional Error. After compiling the Chisel code into Verilog, the Verilog code is passed to the simulator as the Device Under Test (DUT). In addition to the DUT, the simulation process includes a reference module and a series of functional points. Each functional point consists of input stimuli and the expected output. During simulation, input stimuli are applied to both the DUT and the reference module, and their outputs are compared. If the outputs match, the test passes; otherwise, it fails. For all failed test points, we extract the input signals, the expected output signals, and the actual obtained signals to create an error list, which is used as feedback.

Feedback for the two aforementioned error types is provided to the LLM alongside the Chisel code for reflection. The LLM generates a revision plan that, for each error, includes the error location, a cause analysis, and specific solutions. Fig. 3 also illustrates a revision plan based on the compiler feedback shown in the figure. The revision plan guides the LLM in making adjustments to the Chisel code.

C. Non-progress Loop & Escape Mechanism

During the iterative process of code optimization through reflection, LLMs may fall into a *non-progress loop*, i.e., the LLM repeatedly cycles through the same set of errors without resolving them, and is unable to break free from the error cycle. We use Fig. 4 to illustrate this problem. The code error lies within the `switch` block where not all possible cases of `y` are handled, leading to

TABLE II: Common syntax errors in LLM-generated Chisel code and corresponding compiler feedback.

Description	Incorrect	Corrected	Compiler Feedback
A. Structural Errors			
1. Misspelling, unmatched parentheses, etc.	<code>val signal=Wire(...) signal:=0.U</code>	<code>val signal=Wire(...) signal:=0.U</code>	Value signal is not a member. Did you mean signal?
2. Mixed usage of Chisel and Scala syntax.	<code>out := in.asInstanceOf[SInt]</code>	<code>out := in.asSInt</code>	class chisel3.UInt cannot be cast to class chisel3.SInt.
3. Incorrect invocations of functions or methods.	<code>val r=Seq.fill(5)(...) r(0, 2)</code>	<code>val r=Seq.fill(5)(...) r(2)</code>	Too many arguments. Found 2, expected 1 for method apply:(i:Int)
B. Signal Definition, Usage and Typing Errors			
1. Incorrect definition of clock or reset signals.	<code>val rst = IO(Input(Reset()))</code>	<code>val rst = IO(Input(Bool()))</code>	A port rst with abstract reset type was unable to be inferred by InferResets.
2. Failure to encapsulate interface signals within IO().	<code>val clk=Input(Clock())</code>	<code>val clk=IO(Input(Clock))</code>	Clock must be hardware, not a bare Chisel type. Perhaps you forgot to wrap it in Wire(_) or IO(_)?
3. Wire signal not (fully) initialized.	<code>when (io.in) {w:=0.U} // ?</code>	<code>when (io.in) {w:=0.U} otherwise {w:=1.U}</code>	Reference w not fully initialized.
4. Bundle connection mismatch.	<code>val a = OneBdl(...) val b = AnotherBdl(...) a := b</code>	<code>// assignment not allowed</code>	Connection between sink (... OneBdl) and source (... AnotherBdl) failed: .cSource Record missing field (c).
5. Signal type mismatch.	<code>val cnt=oks. reduce(_ +& _)</code>	<code>val cnt=oks.map(_ asUInt). reduce(_ +& _)</code>	found: chisel3.Bool required: chisel3.UInt
6. Unsupported signal type conversion or casting.	<code>invertedClk := (~ clk.asUInt).asClock</code>	<code>// unsupported</code>	Value asClock is not a member of chisel3.UInt.
7. Out-of-bounds access on array-type signal.	<code>vector(-1) := 0.U</code>	<code>vector(0) := 0.U</code>	-1 is out of bounds. (min 0, max 3)
C. Miscellaneous Errors			
1. Incorrect Clock Domain * (in multi-clock design)	<code>val out=RegNext(in)</code>	<code>val out=withClock(clk) { RegNext(in) }</code>	No implicit clock.
2. Combinational Loop	<code>val a=Wire(...) a := a + 1.U</code>	<code>// remove loop</code>	Detected combinational cycle in a FIRRTL module. Sample path: {a ← a_T_1 ← ... ← a_T ← ... ← a}.

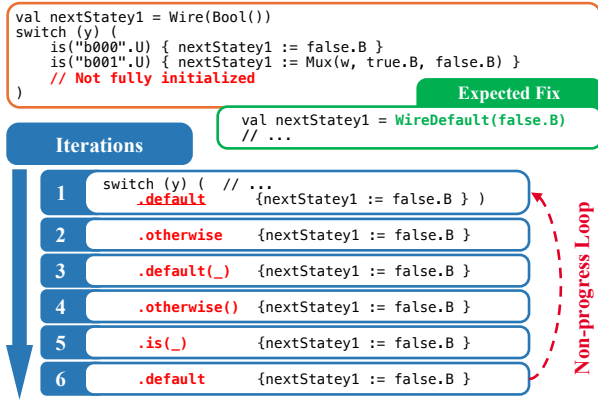


Fig. 4: Example of non-progress loop. The LLM is expected to provide default values when defining the signal to ensure fully initialization. However, it repeatedly attempts to add default statements within the switch block.

nextState1 being uninitialized when y takes unspecified values. The correct approach to fix this is to use WireDefault when defining the signal to provide a default value. However, in the first reflection iteration, the LLM tried to add a default case to the switch block incorrectly, as Chisel's switch does not support any default case syntax. Subsequently, in each iteration, the LLM fell into the trap of repeatedly attempting to add a default case to the switch block, resulting in a non-progress loop.

The non-progress loops happen mainly due to the greedy strategy during reflection and the limitations of the LLM's context. In the example in Fig. 4, after the first iteration fails, the compiler issues an

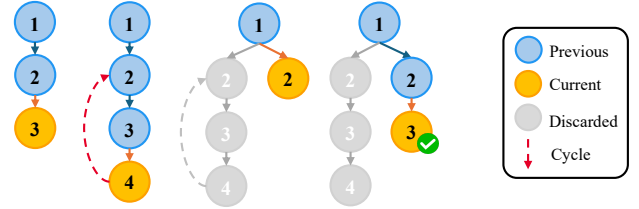


Fig. 5: Illustration of escape mechanism.

error message indicating unsupported syntax in the switch block. Affected by this error, the reviewer focuses solely on correcting issues inside the switch block in subsequent attempts, ignoring the true location of the error which can only be captured from a global perspective. Additionally, the LLM has a limited context length, which means that as the number of iterations increases, early error experiences are completely forgotten, leading the LLM to return to its previous state after multiple attempts.

To address the non-progress loop issue, we propose an *escape* mechanism that monitors the LLM's iterative process for potential cycles. During each iteration, the trace and the current feedback are provided to the Inspector for analysis to detect any cycles. Specifically, the feedback is compared with each entry in the trace. If the current feedback contains an error at the same location as a previous entry's error, and the LLM determines that the causes of these errors are identical, then all iterations between these two points form a non-progress loop. In such cases, the LLM discards all iterations involved in the loop. The Reviewer then regenerates a revision plan starting from the step immediately preceding the loop. With the inherent diversity, the LLM is expected to break out of the

TABLE III: ReChisel performance

Metric	LLMs	Success Rate (%)			
		$n =$			
		0	1	5	10
Pass@1	GPT-4 Turbo	45.54	52.11	67.61	73.24
	GPT-4o	45.07	56.81	73.24	77.46
	GPT-4o mini	11.27	16.43	31.46	40.38
	Claude 3.5 Sonnet	33.33	63.38	80.28	84.98
	Claude 3.5 Haiku	26.29	56.34	79.81	84.51
Pass@5	GPT-4 Turbo	61.97	68.54	80.28	83.10
	GPT-4o	65.26	75.59	83.10	85.45
	GPT-4o mini	28.64	37.56	54.93	62.91
	Claude 3.5 Sonnet	52.58	77.46	91.08	92.49
	Claude 3.5 Haiku	52.11	76.53	90.14	91.08
Pass@10	GPT-4 Turbo	66.20	72.77	84.04	85.92
	GPT-4o	70.89	79.81	85.92	88.73
	GPT-4o mini	36.62	45.54	61.03	67.61
	Claude 3.5 Sonnet	59.62	83.10	92.02	93.43
	Claude 3.5 Haiku	58.69	82.63	91.55	92.96

* n represents the maximum allowed number of iterations.

** The top-1 and top-2 entries in each metric and n group is highlighted using dark and light green shading, respectively.

loop and potentially find the correct fix method. Fig. 5 illustrates this escape mechanism.

V. EVALUATION

This section is organized as follows: Section V-A provides the experimental setup. Section V-B evaluates the performance of ReChisel. Section V-C compares ReChisel with AutoChip, an LLM-based automatic Verilog code generation system. Finally, Section V-D presents a case study to demonstrate the workflow of ReChisel.

A. Experimental Setup

Benchmark. The evaluation is conducted at the module level, where each test case comprises a module specification that includes a functional description and definitions of I/O signals. Our benchmark specifications are sourced from the VerilogEval’s Spec-to-RTL [21], AutoChip’s HDLBits [9], and RTLLM [17] dataset. While most test cases align with our evaluation criteria, some are invalid or incompatible. In particular, issues arise when: 1) the case requires using certain Verilog features such as the parameterization², which are not supported by Chisel. As a result, it is impossible to write Chisel code that compiles into Verilog code incorporating these features; 2) the correctness of generated code cannot be verified due to missing or incorrect reference code; or 3) the case is designed specifically for Verilog code debugging or completion and cannot be adapted for Chisel code generation tasks. After filtering out these cases, we finalized a set of 216 valid test cases.

Model. Five mainstream LLMs are selected for evaluation: GPT-4 Turbo (version: 2024-04-09), GPT-4o (version: 2024-08-06), GPT-4o mini (version: 2024-07-18), and Claude 3.5 Sonnet (version: 2024-10-22), Claude 3.5 Haiku (version: 2024-10-22).

Evaluation Parameters. LLMs are probabilistic, meaning the same prompt can generate different outputs in multiple attempts. Therefore, each case is tested ten times, and the Pass@ k [5] metric is calculated. The LLM is evaluated in its default configuration without manual adjustments to parameters such as temperature and top- p . The maximum number of iterations is capped at 10.

B. Evaluation on ReChisel

Table I demonstrates the performance improvements achieved by ReChisel. The results indicate that ReChisel significantly enhances

²Chisel employs parameterized module generators as a replacement for the `#(parameter ...)` mechanism in Verilog.

TABLE IV: Performance comparison of ReChisel and AutoChip.

Metric	LLMs	Success Rate (%)	
		ReChisel	AutoChip
Pass@1	GPT-4 Turbo	73.24	79.81
	GPT-4o	77.46	78.40
	Claude 3.5 Sonnet	84.98	91.08
Pass@5	GPT-4 Turbo	83.10	87.79
	GPT-4o	85.45	84.51
	Claude 3.5 Sonnet	92.49	96.71
Pass@10	GPT-4 Turbo	85.92	89.20
	GPT-4o	88.73	87.79
	Claude 3.5 Sonnet	93.43	97.65

the performance of LLMs in generating Chisel code, with improvements ranging from 10% to 50%. The four LLMs—GPT-4 Turbo, GPT-4o, Claude 3.5 Sonnet, and Claude 3.5 Haiku—exhibit very similar peak performances, while GPT-4o mini’s performance is notably inferior. Among these, Claude 3.5 Sonnet achieved the highest final success rates, with 84.98% under Pass@1 and 93.43% under Pass@10. Notably, although Claude 3.5 Haiku is much more lightweight than Sonnet, its performance is nearly on par with that of Sonnet, and it even surpasses GPT-4o and GPT-4 Turbo.

Fig. 6 further illustrates how the success rates of LLMs change as the number of iterations increases. As shown in the figure, the two Claude LLMs demonstrate much stronger reflection capabilities. Taking Sonnet as an example, under the Pass@1 metric, its baseline accuracy ($n = 0$) is only 33%, significantly lower than that of GPT-4 Turbo or GPT-4o. However, after a few iterations, its performance improves substantially, eventually surpassing GPT-4 Turbo and GPT-4o at $n = 10$. After about four iterations, all LLMs except GPT-4o mini can reach a performance plateau. In contrast, GPT-4o mini’s performance improves more slowly as the number of iterations increases. After reaching the plateau, about 10% of cases still cannot be generated correctly, highlighting the inherent limitations of LLMs.

We also examine how the number of syntax and functional errors changes with increasing iterations. In Fig. 7, we present the proportions of syntax and functional errors for GPT-4o under the Pass@1 metric across each iteration. Compared to the baseline, ReChisel reduces both syntax and functional errors. However, we observed that in some cases (e.g., from the 9th to the 10th iteration in the figure), the proportion of syntax errors increased. This suggests that when the LLM fixes functional errors, it may inadvertently reintroduce syntax errors.

C. Performance Comparison of ReChisel and AutoChip

To further investigate the questions raised in Section III, we compare ReChisel with AutoChip [26]. AutoChip is an LLM-based agentic system designed for automatic *direct* Verilog code generation. Both systems are evaluated using the same benchmarks and parameters given in Section V-A. We select OpenAI’s GPT-4o, GPT-4 Turbo, and Claude 3.5’s Sonnet models for comparison. For both systems, the maximum number of iterations is set to 10. The results presented in Table III demonstrate that ReChisel enhances the LLM’s Chisel code generation performance to a level almost comparable with Verilog generation. Specifically, for the GPT-4o model, ReChisel outperforms AutoChip in both Pass@5 and Pass@10 metrics. These results demonstrate the great potential of using Chisel for hardware code generation.

D. Case Study

Fig. 8 illustrates the ReChisel’s reflection workflow through a case study. This case corresponds to the `Vector5` test case from AutoChip’s HDLBits benchmark, utilizing the GPT-4o model. The figure

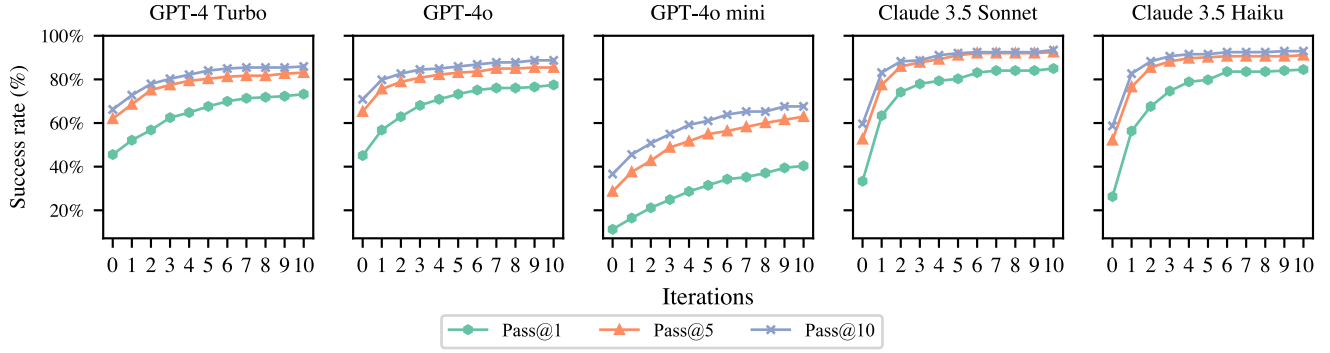


Fig. 6: Relationship between success rate and number of iterations for different LLMs.

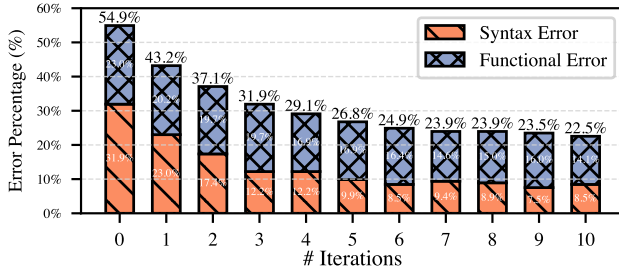


Fig. 7: Proportion of syntax and functional errors across iterations. Example of GPT-4o under Pass@1 metric.

illustrates the specifications and demonstrates how an LLM iteratively refines its output to progressively generate syntactically correct and functionally accurate Chisel code through reflection. The process was completed in three iterations. During the first two iterations, the code encountered syntax errors. In the first iteration, the LLM introduced intermediate variables to address multiple assignment issues, but an incorrect data type choice caused another error. In the second iteration, the LLM used boolean vectors that support bitwise operations, successfully generating syntactically correct code. In the third iteration, the syntactically correct code contained a functional error. The LLM then corrected the loop logic, ultimately producing a functionally correct implementation that passed all tests.

VI. CONCLUSION

In this work, we introduce ReChisel, an LLM-based agentic system designed to enhance the effectiveness of Chisel code generation. ReChisel incorporates a reflection mechanism to iteratively refine the quality of generated code using feedback from compilation and simulation processes, and introduces an escape mechanism to break free from non-progress loops. Experiments across three benchmarks and five mainstream LLMs demonstrate that ReChisel significantly improves the success rate of Chisel code generation, achieving performance comparable to state-of-the-art LLM-based agentic systems for Verilog code generation. This work highlights the potential of Chisel in LLM-assisted hardware design.

VII. ACKNOWLEDGEMENT

This work is supported by the National Key Research and Development Program (Grant No. 2024YFB4405600), the National Natural Science Foundation of China (Grant No. 62472086), the Basic Research Program of Jiangsu (Grants No. BK20243042, BG2024010), and the Start-up Research Fund of Southeast University (Grant No. RF1028624005).

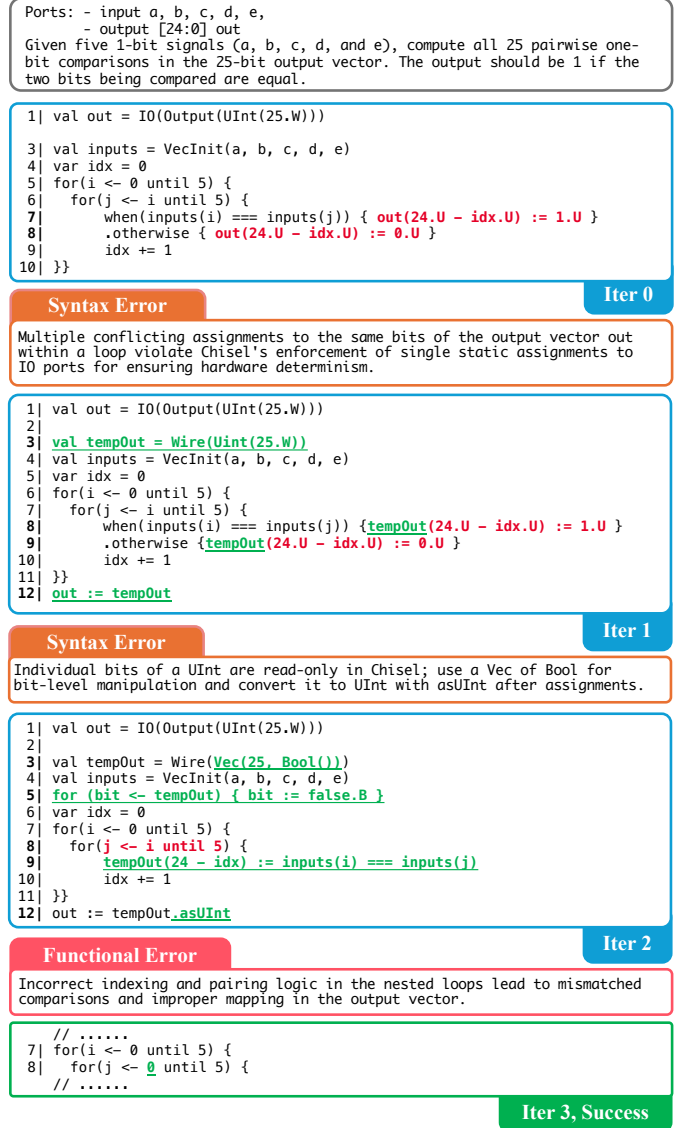


Fig. 8: Case study on reflection. Green underlined codes represent modifications based on feedback from the previous iteration, while red codes indicate syntax or functional errors in the current iteration. The content within the error boxes summarizes the corresponding reviewer plans.

REFERENCES

- [1] K. Asanovic, R. Avizienis, J. Bachrach, S. Beamer, D. Biancolin, C. Celio, H. Cook, D. Dabbelt, J. Hauser, A. Izraelevitz *et al.*, “The rocket chip generator,” *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17*, vol. 4, pp. 6–2, 2016.
- [2] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avizienis, J. Wawrzyniak, and K. Asanović, “Chisel: constructing hardware in a scala embedded language,” in *Proceedings of the 49th Annual Design Automation Conference*, 2012, pp. 1216–1225.
- [3] J. Blocklove, S. Garg, R. Karri, and H. Pearce, “Chip-chat: Challenges and opportunities in conversational hardware design,” in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 2023, pp. 1–6.
- [4] K. Chang, Y. Wang, H. Ren, M. Wang, S. Liang, Y. Han, H. Li, and X. Li, “Chipppt: How far are we from natural language hardware design,” *arXiv preprint arXiv:2305.14019*, 2023.
- [5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [6] M.-S. Contributors, “Magma-si: A matrix accelerator generator,” <https://github.com/merledu/magma-si>, 2024, accessed: 2024-11-15.
- [7] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li, H. Xia, J. Xu, Z. Wu, T. Liu *et al.*, “A survey on in-context learning,” *arXiv preprint arXiv:2301.00234*, 2022.
- [8] H. Genc, A. Haj-Ali, V. Iyer, A. Amid, H. Mao, J. Wright, C. Schmidt, J. Zhao, A. Ou, M. Banister *et al.*, “Gemmini: An agile systolic array generator enabling systematic evaluations of deep-learning architectures,” *arXiv preprint arXiv:1911.09925*, vol. 3, no. 25, pp. 15–17, 2019.
- [9] HDLBits, “Problem sets - hdlbits,” https://hdlbits.01xz.net/wiki/Problem_sets, accessed: 2024-11-11.
- [10] F. Heilmann, C. Brugger, and N. Wehn, “White paper-investigate the high-level hdl chisel,” *Kaiserslautern, Germany*, 2013.
- [11] C.-T. Ho, H. Ren, and B. Khailany, “Verilogcoder: Autonomous verilog coding agents with graph-based planning and abstract syntax tree (ast)-based waveform tracing tool,” *arXiv preprint arXiv:2408.08927*, 2024.
- [12] A. Izraelevitz, J. Koenig, P. Li, R. Lin, A. Wang, A. Magyar, D. Kim, C. Schmidt, C. Markley, J. Lawson, and J. Bachrach, “Reusability is firrtl ground: Hardware construction languages, compiler frameworks, and transformations,” in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, Nov 2017, pp. 209–216.
- [13] P. Lennon and R. Gahan, “A comparative study of chisel for fpga design,” in *2018 29th Irish Signals and Systems Conference (ISSC)*. IEEE, 2018, pp. 1–6.
- [14] P. S. Li, A. M. Izraelevitz, and J. Bachrach, “Specification for the firrtl language,” *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-9*, Feb 2016. [Online]. Available: <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-9.html>
- [15] M. Liu, N. Pinckney, B. Khailany, and H. Ren, “VerilogEval: Evaluating large language models for verilog code generation,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–8.
- [16] S. Liu, W. Fang, Y. Lu, J. Wang, Q. Zhang, H. Zhang, and Z. Xie, “Rtl-coder: Fully open-source and efficient llm-assisted rtl code generation technique,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [17] S. Liu, Y. Lu, W. Fang, M. Li, and Z. Xie, “Openllm-rtl: Open dataset and benchmark for llm-aided design rtl generation(invited),” in *Proceedings of 2024 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. ACM, 2024.
- [18] T. Liu, Q. Tian, J. Ye, L. Fu, S. Su, J. Li, G.-W. Wan, L. Zhang, S.-Z. Wong, X. Wang *et al.*, “Chatchisel: Enabling agile hardware design with large language models,” in *2024 2nd International Symposium of Electronics Design Automation (ISED)*. IEEE, 2024, pp. 710–716.
- [19] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, “RtlLm: An open-source benchmark for design rtl generation with large language model,” in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 722–727.
- [20] H. Pearce, B. Tan, and R. Karri, “Dave: Deriving automatically verilog from english,” in *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD*, 2020, pp. 27–32.
- [21] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany, “Revisiting verilogEval: Newer llms, in-context learning, and specification-to-rtl tasks,” *arXiv preprint arXiv:2408.11053*, 2024.
- [22] M. Schoeberl, *Digital design with chisel*. Kindle Direct Publishing, 2019.
- [23] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflection: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [24] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, “Benchmarking large language models for automated verilog rtl code generation,” in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2023, pp. 1–6.
- [25] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, “Verigen: A large language model for verilog code generation,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 3, pp. 1–31, 2024.
- [26] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, “Autochip: Automating hdl generation using llm feedback,” in *Proceedings of the 61st Design Automation Conference (DAC)*, San Francisco, CA, 2024.
- [27] Y. Tsai, M. Liu, and H. Ren, “Rtlfixer: Automatically fixing rtl syntax errors with large language model,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [28] X. Wang, G.-W. Wan, S.-Z. Wong, L. Zhang, T. Liu, Q. Tian, and J. Ye, “Chatcpu: An agile cpu design & verification platform with llm,” in *61st ACM/IEEE Design Automation Conference (DAC’24)*, 2024, p. 6.
- [29] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems*, vol. 35, pp. 24824–24837, 2022.
- [30] S.-Z. Wong, G.-W. Wan, D. Liu, and X. Wang, “Vgv: Verilog generation using visual capabilities of multi-modal large language models,” in *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 2024, pp. 1–5.
- [31] Y. Xu, Z. Yu, D. Tang, G. Chen, L. Chen, L. Gou, Y. Jin, Q. Li, X. Li, Z. Li *et al.*, “Towards developing high performance risc-v processors using agile methodology,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1178–1199.
- [32] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [33] X. Yao, H. Li, T. H. Chan, W. Xiao, M. Yuan, Y. Huang, L. Chen, and B. Yu, “Hdldebugger: Streamlining hdl debugging with large language models,” *arXiv preprint arXiv:2403.11671*, 2024.
- [34] X. Yao, Y. Wang, X. Li, Y. Lian, R. Chen, L. Chen, M. Yuan, H. Xu, and B. Yu, “Rtlrewriter: Methodologies for large models aided rtl code optimization,” *arXiv preprint arXiv:2409.11414*, 2024.
- [35] Y. Zhang, Z. Yu, Y. Fu, C. Wan, and Y. C. Lin, “Mg-verilog: Multi-grained dataset towards enhanced llm-assisted verilog generation,” in *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 2024, pp. 1–5.
- [36] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, “Sonicboom: The 3rd generation berkeley out-of-order machine,” May 2020.